HW6 Ray Tracing — Technical Specs

Daniel Oni · CS 7382/5382 Computer Graphics · Spring 2026

Rendered scene: a checkered ground plane with a procedurally textured earth sphere, a clear glass sphere, a polished silver sphere, a copper box built from six planar quads, and a large field of random spheres mixing diffuse, metal, glass, and hollow-glass materials.

Required technical parameters

Parameter	Value
Image resolution	800 x 450 pixels (16:9)
Samples per pixel	300
Maximum bounce depth	50
Render time (on my CPU)	19 minutes 5 seconds
Camera lookfrom	(13, 2, 3)
Camera lookat	(0, 0, 0)
Camera up vector	(0, 1, 0)
Vertical field of view	20 degrees
Defocus angle	0.6 degrees
Focus distance	10.0

The image of the scene was produced on my own machine using the command `.\rt.exe > image.ppm` after compiling main.cc with the Visual Studio 2019 compiler at `/O2 /std:c++17 /EHsc`. The resulting PPM file was converted to JPG for submission. Full build instructions for Windows, macOS, and Linux are included in the submission as build_and_run.md.

How I coded it

I started from the code structure described in *Ray Tracing in One Weekend*. The book builds up a path tracer in short steps, and I followed those steps closely for the core of the program: vectors, rays, the hittable interface, sphere intersection, materials, and the camera. Once I had the baseline cover image working, I added three extensions on top of it so that the final image would go beyond what you get by just running the book code unchanged.

The three extensions are a procedural checker texture on the ground, an image-based earth texture on one of the foreground spheres, and a completely new box primitive that I built out of six planar quads. Each of those required small but real changes to the existing classes, which I describe in the sections below.

The box primitive (custom geometry)

For the custom geometry extension I wrote a new class called `quad`. A quad is a flat parallelogram, defined by a corner point `Q` and two edge vectors `u` and `v`. Any point on the quad can be written as `P = Q + a*u + b*v` where `a` and `b` are each in the range [0, 1].

The surface normal of a quad is the same at every point, since the quad is flat. I compute it once in the constructor as `normalize(cross(u, v))`. This is the same right-hand-rule cross product we used in the lighting lectures to derive the plane normal from two edge vectors. Because the orientation of the normal depends on the order of `u` and `v`, I was careful when building the six sides of the box to choose edge vectors so that every side points outward. The hit record still goes through set_face_normal so that the front/back distinction works the same way it does for spheres, which means dielectrics and metals behave correctly no matter which side of the quad gets hit.

Ray-quad intersection is done in two steps. First I solve the ray-plane equation to find `t`, which gives the hit point on the infinite plane the quad sits on. Then I project the hit point back into the local (a, b) coordinates and test whether both are in [0, 1]. If they are, the hit is inside the quad and I pass those (a, b) values through as the u and v texture coordinates of the hit record. If not, the ray missed the quad even though it hit the plane.

The box itself is built in a small helper function that takes two opposite corner points and a material, then emits six quads wrapped in a hittable_list. One nice thing about building a box from quads this way is that each face already has its own material and normal, so I did not need a separate box class with its own intersection code. The hittable_list just walks its children and returns the closest hit, exactly like it does for the rest of the scene.

Textures

Adding textures required one small change to the hit record. I added two doubles, `u` and `v`, so that a material's scatter function could ask the texture what color to return at the hit point. The sphere class now fills in those values from a standard spherical mapping (longitude for u, latitude for v), and the quad class fills them in from the local (a, b) coordinates described above.

The ground is covered by a procedural checker_texture. It does not need an image file. It looks at the world-space x, y, and z of the hit point, floors them after scaling, and picks one of two colors based on whether the sum of those floors is even or odd. That is why the checker stays sharp no matter how far from the camera it is. There is no texture resolution to run out of.

The earth sphere uses an image_texture that reads a JPG from disk with stb_image. I did not want to use NASA's real blue marble image, partly because I wanted the whole project to be self-contained and partly because I wanted to see how hard it would be to generate an earth-like planet procedurally. I wrote a small Python script called make_earthmap.py that uses layered value noise to build continents, biome variation, polar ice caps, and a few cloud wisps, and saves the result as earthmap.jpg. The resulting planet is not any real place on earth, but when it is mapped onto a sphere with the standard spherical UV formula it reads as a clearly earthlike world. The script is included in the submission and the random seed is fixed so the output is reproducible.

Computational tradeoffs

Why 300 samples per pixel

Every pixel in this image is the average of 300 independent rays, each randomly jittered inside the pixel footprint and, because the camera has a defocus angle, originating from a random point on a small disk around the camera center. That single number controls almost all of the visual quality: anti-aliasing on the edges of objects, smoothness on lambertian surfaces, the graininess of fuzzy metal reflections, and the cleanness of the defocus blur.

I tried lower values first. At 32 samples per pixel the scene is already recognizable but the diffuse spheres look dusty and the glass sphere has obvious speckle inside. At 100 samples per pixel most of the noise is gone. At 300 samples per pixel even the fuzzy metal box sides and the soft defocus in the back of the scene look smooth. Doubling the samples cuts the remaining noise roughly in half, so the returns get smaller past this point and it was not worth the extra minutes to go to 500 or more.

Why maximum depth 50

The max_depth value caps how many times a ray is allowed to bounce before the path tracer gives up and returns black. For a scene with this much glass, a ray can easily refract into a sphere, refract out, hit another glass sphere, refract through that one, and so on. If the cap is too low, the insides of glass objects turn dark because the ray ran out of bounces before it could escape the scene and pick up the sky color. I landed on 50 because it is comfortably larger than any chain of refractions I would reasonably expect, and the cost of a higher cap is small since most rays terminate early anyway.

Why there is no BVH

The book introduces a bounding volume hierarchy in its second and third books, and I considered pulling it in. In the end I decided not to. The scene has roughly 480 spheres and 6 quads, and the hittable_list just checks every primitive against every camera ray. That is linear in the number of objects, which is the classic cost of a naive ray tracer. A BVH would turn that into roughly logarithmic cost and bring the render down from minutes to seconds.

The tradeoff for me was simplicity against speed. Without a BVH, the code is small enough that I can read through any of the source files and understand exactly what is happening on every bounce. Adding a BVH would have introduced an acceleration structure, an axis-aligned bounding box type, and a tree build phase, all of which would have made the code harder to follow. I accepted the slower render because this is a one-time image rather than something interactive. If I were rendering an animation or iterating quickly on lighting, a BVH would be worth the extra code. For a single final image I would rather wait.

Why the hollow glass trick

A small number of the random spheres are built as two spheres at the same center, one with a positive radius and one slightly smaller with a negative radius. The negative-radius sphere is a trick from the book: the geometry is the same, but the surface normal is flipped, so the dielectric material treats the inside as being filled with air. The visual effect is a hollow glass bubble with a visible inner surface, which adds a little extra optical interest to the scene without any new code. It is one of my favorite details in the image.

Why the camera is where it is

The camera sits at (13, 2, 3) looking at the world origin with a 20-degree vertical field of view. A narrow field of view and a fairly distant camera keeps the foreground perspective gentle, which lets the random spheres line up in the midground without exploding in size. The low y value of 2 puts the camera just above the tops of the small spheres, so you see across the scene rather than down onto it. The defocus angle of 0.6 degrees with a focus distance of 10 matches up so the three hero objects at roughly x=0 are in sharp focus and the far corners of the sphere field go soft, which gives the image depth.